CODE REVIEW FINAL ASISTIDO POR IA — TPI Food Store (2ª entrega, U4–U8)

- **Fecha:** 2026-11-10 · **Revisor:** OpenCode (asistente de IA) · **Decisión final:** equipo (humano), registrada por hallazgo más abajo.
- **Alcance:** la totalidad de los scripts de la cursada (41 scripts, ~153 KB): Parcial (8), TP2 (2), TP3 (4), TP4 (4), TP5 (2), TP6 (4), U5 (4 SQL), U6 (6 PowerShell), U7 (4: JS/SQL/Python), U8 (3 SQLite).
- **Método:** lectura línea por línea de cada script; los hallazgos se **verificaron contra la base real** (copias de prueba o consultas de solo lectura sobre el catálogo) antes de ser aceptados; nada se afirma sin verificación. Amplía el review parcial del TP8 (consigna 3.c: 3 hallazgos, ver CR-10) al objetivo oficial de la 2ª entrega ("code review final asistido por IA de todos los scripts de la cursada, con la decisión humana documentada sobre cada hallazgo").

Resumen

Resultado	Cantidad
Scripts revisados	41
Hallazgos verificados nuevos (CR-01…CR-08)	8
Hallazgos ya cerrados (CR-09 U6, CR-10 TP8)	2 grupos
Scripts sin hallazgos	24 (correctos y con sus evidencias)
Correcciones aplicadas en esta entrega	1 en base (CR-03) + scripts versionados con el fix

---

CR-01 — `TP3\carga_masiva.sql:105` — Bug real del seed (SEVERIDAD ALTA)

Hallazgo. El generador del `subtotal` de `detalle_pedido` usa un número aleatorio **independiente** del usado para `cantidad`:

```
1 + floor(random() * 9)::int          AS cantidad,
pr.precio,
pr.precio * (1 + floor(random() * 9)::int) AS subtotal   -- <-- random() NUEVO, no el de cantidad
```

El comentario del script (líneas 78-79 y 94) afirma "subtotal consistente / subtotal = cantidad * precio_unitario", que es **falso**: la probabilidad de coincidir es 1/9 por fila.

Verificación ejecutada. - Conteo en el origen y en las restauradas: **533.524 filas** con `abs(cantidad*precio_unitario - subtotal) > 0.01` (≈ 8/9 de 601.205 — coincide con la probabilidad teórica). - El hallazgo ya había surgido en el simulacro U6 (informe post-incidente) como desvío de calidad de datos; este review le encontró la **causa raíz en el script**. - El `DISABLE TRIGGER ALL` de la carga (decisión de performance documentada en el propio script) desactivó también `trg_detalle_verificar_subtotal`, la red de seguridad que habría detectado el bug en el momento. - Consecuencia verificada: esas 533.524 filas quedan **congeladas** — cualquier `UPDATE` (aunque no toque importes) re-dispara el trigger y falla: `ERROR: El subtotal (14530.71) no coincide con cantidad (7) x precio_unitario (4843.57). Resultado esperado: 33904.99`. Todos los reportes basados en `sum(subtotal)` de TP3–TP6 arrastran el desvío.

Decisión humana (equipo): documentar como bug del seed; **no recargar la base** (rompería las evidencias de planes de ejecución medidos de TP3/TP4/TP5/TP6). El fix del generador queda versionado abajo para cargas futuras.

```
-- FIX VERSIONADO (generador corregido para futuras cargas):
WITH linea AS (
    SELECT g, linea,
           1 + floor(random() * 9)::int AS cantidad,          -- UNA sola tirada
           1 + ((g * 7 + linea) % 50000)::int AS producto_id
    FROM generate_series(1, 200000) g CROSS JOIN generate_series(0, 2) linea
)
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
SELECT l.g, l.producto_id, l.cantidad, pr.precio, l.cantidad * pr.precio  -- subtotal derivado
FROM linea l CROSS JOIN LATERAL (SELECT precio FROM producto WHERE id = l.producto_id) pr;
```

CR-02 — `Parcial\triggers_restricciones.sql` (TP2) — Regla de stock solo en INSERT (SEVERIDAD MEDIA)

Hallazgo. `trg_detalle_descontar_stock` es `BEFORE INSERT` únicamente: un `UPDATE` de `cantidad` (manteniendo el subtotal coherente para pasar `trg_detalle_verificar_subtotal`) **no valida ni ajusta stock**.

Verificación ejecutada (copia de prueba, transacción con `ROLLBACK`):

```
BEGIN;
UPDATE detalle_pedido SET cantidad = 999999, subtotal = 999999 * precio_unitario WHERE id = 1;
-- UPDATE 1  (aceptado)
SELECT stock FROM producto WHERE id = (SELECT producto_id FROM detalle_pedido WHERE id = 1);
-- stock = 192  (intacto: cantidad 999999 sin stock que la respalde)
ROLLBACK;
```

Decisión humana (equipo): documentar con fix versionado; no aplicar sobre la base de la cursada (cambiaría el comportamiento medido en las evidencias del TP2). La consigna del TP2 decía literalmente "stock **al crear** detalle", por lo que la implementación INSERT-only respeta lo pedido; el gap es de evolución de negocio.

```
-- FIX VERSIONADO (trigger con lógica delta, para INSERT OR UPDATE):
CREATE OR REPLACE FUNCTION fn_descontar_stock()
RETURNS TRIGGER AS $$
DECLARE v_delta INTEGER := NEW.cantidad - COALESCE(OLD.cantidad, 0);
BEGIN
    UPDATE producto SET stock = stock - v_delta
     WHERE id = NEW.producto_id AND stock >= v_delta;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Stock insuficiente para el producto id=%', NEW.producto_id;
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;
-- y el trigger: BEFORE INSERT OR UPDATE ON detalle_pedido
```

CR-03 — `U5_admin_seguridad\sql\roles.sql:170-171` — DELETE en default privileges contradice R7 (SEVERIDAD MEDIA) — **CORREGIDO**

Hallazgo. `ALTER DEFAULT PRIVILEGES ... GRANT INSERT, UPDATE, DELETE ON TABLES TO rol_app_escritura` otorga `DELETE` en tablas **futuras**, contradiciendo la política de baja lógica R7 y los grants explícitos de la Parte A (que solo dan `INSERT, UPDATE` sobre `pedido` / `detalle_pedido`).

Verificación ejecutada (catálogo del origen, solo lectura): `pg_default_acl` mostraba `rol_app_escritura=awd`.

Corrección aplicada (decisión humana: corregir script + base): 1. `roles.sql` versionado con el fix y su justificación. 2. En `food_store_tp3` (metadata-only, reversible, no toca datos ni tablas existentes): `ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public REVOKE DELETE ON TABLES FROM rol_app_escritura;` 3. Verificación post-fix: `pg_default_acl` → `rol_app_escritura=aw` (antes `awd`). Evidencia completa antes/después en `evidencias/cr03_default_privileges_20261110.txt`.

CR-04 — `schema.sql` + TP3/TP4 — Sobreindexación por evolución (SEVERIDAD MEDIA-BAJA)

Hallazgo. Los índices simples del TP1 quedaron prefijo-cubiertos por índices compuestos/cover agregados después:

- `idx_detalle_pedido (pedido_id)` ⊂ prefijo de `uq_detalle_pedido_producto (pedido_id, producto_id)` y de `idx_detalle_cover_pedido_prod (pedido_id, producto_id) INCLUDE (subtotal)` (TP4).
- `idx_detalle_producto (producto_id)` ⊂ prefijo de `idx_detalle_producto_cover (producto_id) INCLUDE (cantidad, subtotal)` (TP3).

Verificación ejecutada: `pg_indexes` sobre `detalle_pedido` — coexisten PK + 5 índices; los dos simples son redundantes hoy (el motor resuelve las mismas búsquedas por prefijo de los compuestos).

Decisión humana (equipo): mantener en la base de la cursada — los planes de ejecución medidos y documentados (evidencias TP3/TP4) son parte de la entrega; la limpieza queda registrada como acción de mantenimiento futuro (mismo criterio disciplinado que el índice descartado del TP5, `spec_indice_descartado_producto_categoria.md`).

CR-05 — `Parcial\objetos.sql` — `fn_total_cliente` y vistas sin filtro `eliminado` (SEVERIDAD BAJA, latente)

Hallazgo. `fn_total_cliente`, `vista_pedidos_cliente` y `vista_detalle_pedido` no filtran `pedido.eliminado` / `detalle_pedido.eliminado` (columna agregada por TP6 con posterioridad a la creación de estos objetos): contarían pedidos anulados en totales e historiales.

Verificación ejecutada: hoy `pedido.eliminado=TRUE` tiene **0 filas** y `detalle_pedido.eliminado=TRUE` tiene **0 filas** → sin impacto actual; el riesgo existe solo si se anulan pedidos.

Decisión humana (equipo): nota de limitación documentada (aceptar); en una iteración de negocio se alinearían los objetos a la semántica de baja lógica del TP6.

CR-06 — TP2 `fn_descontar_stock` — Check-then-act no atómico (SEVERIDAD BAJA)

Hallazgo. El trigger hace `SELECT stock` y luego `UPDATE` sin lock: bajo dos INSERT concurrentes del mismo producto (READ COMMITTED), ambos checks pueden ver el mismo stock y el segundo `UPDATE` aplica sobre la fila ya descontada.

Verificación (análisis + esquema): el `CHECK (stock >= 0)` de `producto` (schema.sql) **impide el stock negativo** — la transacción que llega tarde falla con violación de constraint (error genérico) en lugar del mensaje de negocio "Stock insuficiente...". La integridad no se compromete; degrada la semántica del error.

Decisión humana (equipo): mejora futura opcional (el fix delta de CR-02 la resuelve de paso: `UPDATE ... WHERE stock >= v_delta` es atómico). No se aplica sobre la base de evidencias.

CR-07 — `Parcial\objetos_avanzados.sql` — Trigger de auditoría de precios con alcance excesivo (SEVERIDAD BAJA, latente)

Hallazgo. `trg_auditar_ajuste_precios` es `AFTER UPDATE ON producto FOR EACH STATEMENT` sin discriminación de columna: si quedara activo, **cada descuento de stock de cada venta** (el `UPDATE producto SET stock = stock - x` del trigger del TP2) dispararía una fila espuria en `auditoria_precios`. La demo del script termina en `ROLLBACK`, pero el `CREATE TRIGGER` está **fuera** de la transacción demo.

Verificación ejecutada: hoy NO está aplicado en `food_store_tp3` (`pg_trigger` → no existe; `to_regclass('auditoria_precios')` y `producto_extra` → NULL; la base tiene 13 tablas, sin estos objetos). El riesgo es solo si se re-ejecuta el script tal cual.

Decisión humana (equipo): nota documentada; para re-ejecuciones, envolver el DDL en la transacción demo o agregar `DROP TRIGGER` al final / `WHEN (OLD.precio IS DISTINCT FROM NEW.precio)`.

CR-08 — `U8_tablet_cierre\movil\semaforo.sql` — Solicitud PENDIENTE sin ítems (SEVERIDAD BAJA, teórico)

Hallazgo. El `JOIN item_borrador i ON i.id_local = c.id_local` excluye del semáforo a una solicitud `PENDIENTE` cuyo borrador no tiene ítems: quedaría `PENDIENTE` indefinidamente.

Verificación (análisis): en el flujo real del TP8, todo borrador ingresado a la cola tiene ≥ 1 ítem (los casos C2–C6 se probadon con ítems). No reproducible en los casos ejecutados.

Decisión humana (equipo): nota de diseño aceptada (opcional futuro: `LEFT JOIN` + tratar el caso "sin ítems" explícito).

CR-09 — U6: 6 scripts PowerShell — Hallazgos F1–F5 (ya cerrados)

Hallados, corregidos y **re-probados durante la corrida real de U6** (2026-11-09, ver `TPI2_U6_mantenimiento/evidencias/RESUMEN_CORRIDA_U6_20261109.md`):

Fallo	Causa raíz	Corrección verificada
F1	Operador `??` inexistente en PowerShell 5.1	Defaults reescritos con `if/else`; parseo 6/6 OK

Fallo	Causa raíz	Corrección verificada
F2	`$ErrorActionPreference="Stop"` + stderr de herramientas nativas → `NativeCommandError`	Cambiado a `Continue`; exit codes explícitos
F3	Función `R` colisionaba con el alias `r` (`Invoke-History`) → **falso "SUPERADAS"** sin ejecutar checks	Renombrada `Invoke-Check`; re-ejecución 3/3 OK
F4	`$PSScriptRoot` vacío en defaults de parámetros bajo `powershell.exe -File` (tarea programada) → exit 1	Resolución de rutas al cuerpo del script; tarea re-registrada OK (`LastTaskResult=0`)
F5	Bug de un comando comparativo ad-hoc (no de los scripts)	Re-hecho con `StartsWith`

CR-10 — Review parcial previo del TP8, consigna 3.c (ya decididos)

Referenciados (no se duplican): `fn_autenticar` sin `search_path` fijo — **aceptado**; `sp_registrar_pedido` deriva producto inexistente en `NULL` → violación NOT NULL — **aceptado** (demo con `CALL` + `ROLLBACK`); eliminación de `idx_pedido_fecha` por redundancia aparente — **descartado con fundamento** (la columna líder `forma_pago` del compuesto no sirve a filtros solo por rango de `fecha`). Detalle en `U8_tablet_cierre/docs/bitacora_ia.md` (Entrada 4) e informe del TP8.

Scripts sin hallazgos (24)

`Parcial\ddl.sql` (== `schema.sql`, hash idéntico verificado), `consultas.sql`, `transacciones_prueba.sql`, `borrado_logico_prueba.sql`, `TP2\test_restricciones.sql` (SAVEPOINTs correctos), `TP3\consultas_lentas.sql`, `parte4_equivalencia.sql` (EXCEPT bidireccionales correctos), `TP4\parte1_consultas.sql`, `parte1_optimizacion.sql`, `parte3_ranking.sql` (equivalencia RANK vs count+1 verificada), `parte3_subconsulta_correlacionada.sql`, `TP5\indices.sql` (specs y descarte documentados), `TP5\views.sql` (vista sin dato sensible, MV con índice único), `TP6\00_setup_ambiente.sql` (idempotente, subtotal coherente en el seed), `TP6\01_tp_fnbc_control_lote.sql`, `TP6\02_tp_desnormalizacion_top_categorias.sql` (auditoría de desync), `TP6\04_migracion_transaccional_reversible.sql` (nuevo, probado esta entrega), `U5\auditoria_test.sql`, `U5\simulacro_incidente.sql`, `U5\usuario_anon.sql`, `U7\migracion_pedidos.js`, `U7\consultas_mongodb.js`, `U7\consultas_postgresql.sql`, `U7\ejecutar_mongodb.py`, `U8\schema_sqlite.sql` (STRICT+FK+centavos), `U8\pruebas_sqlite.sql` (C1/C7 con ROLLBACK).

Notas menores sin acción requerida: `TP3\optimizaciones.sql` crea índices sin `CONCURRENTLY` dentro de transacción (bloquea escrituras durante el build — aceptable en copia de trabajo, patrón deliberado para medir); `U7` migra un subconjunto acotado (5 pedidos) con valores históricos copiados (decisión documentada en `INFORME_U7.md`); `TP6\00_setup_ambiente.sql` usa `ORDER BY random()` en el seed (costo one-shot aceptable en laboratorio).

Protocolo aplicado en este review

- [x] Ningún hallazgo se aceptó solo por lo que dijo la IA: cada uno se verificó contra la base real (conteos, catálogo, o demo con transacción + `ROLLBACK`) o se marcó explícitamente como teórico/latente con su análisis.
- [x] No se enviaron datos personales/sensibles a la IA (las verificaciones usan copias y valores sintéticos).
- [x] La decisión final de cada hallazgo es del equipo humano y quedó registrada arriba (2026-11-10).
- [x] Las correcciones se versionaron en Git junto con este documento.